



Arrays



The Array as an Abstract Data Type

- Although, an array is usually implemented as a **consecutive set of memory locations**, it shows an emphasis on implementation issues.
- Intuitively, an array is a set of pairs: **<index, value>** (correspondence or mapping), such that each **index** has a **value** associated with it.
- When considering an array as an ADT, it often has the operations: **retrieve, store, creating an array**.
- Now we show a **class definition** of the array ADT.

GeneralArray

```
class GeneralArray {
```

```
// a set of pairs <index, value> where index  $\in$  IndexSet which is  
// a finite ordered set of one or more dimensions, for example {(0,0),  
//(0,1),(0,2),(1,0),(1,1),(1,2),(2,0),(2,1),(2,2)} for two dimensions, etc.  
// and the type of value is float.
```

```
public:
```

```
    GeneralArray(int j, RangeList list, float initValue =  
                                                         defaultValue);
```

```
// This constructor creates a j dimensional array of floats;  
// the range of the kth dimension is given by the kth element of list.  
// For each index i in the index set, initValue is associated with i in  
the array.
```

float Retrieve(index i);

// if i is in the index set of the array, return the float value

// associated with i in the array; otherwise throw an exception.

void Store(index i, **float** x);

// if i is in the index set of the array, **replace the old value**

// associated with i by x; otherwise throw an exception.

}; //end of GeneralArray

Note that:

- GeneralArray is **more general** than C++ array as it is more flexible about **the index set**.
 - ✓ The C++ array requires the index set to be a set of **consecutive integers starting at 0**.
 - ✓ Also, C++ **does not check an array index** to ensure that it belongs to the range for which the array is defined.
- But, a general array class would typically be **internally implemented by using C++ array**.
 - ✓ So, it is necessary to be able to map a position in the general array onto a position in the C++ array.
- To simplify the discussion, **we will hereafter use the C++ array**.

Ordered or linear list

Examples:

- ✓ Days of a week: (Sun, Mon, Tue, Wed, Thu, Fri, Sat)
- ✓ Values in a deck of cards: (2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K, A)
- ✓ Years Switzerland fought in World War II: () // empty list

Generally, **an ordered list** is either **empty** or in the form **$(a_0, a_1, \dots, a_{n-1})$** .

Main operations on ordered list:

- (1) **Find the length**, n , of the list.
- (2) **Read** the list from left to right (or right to left)
- (3) **Retrieve** the i th element, $0 \leq i < n$.
- (4) **Store** a new value into the i th position, $0 \leq i < n$.
- (5) **Insert** a new element at position i , $0 \leq i < n$, causing elements numbered $i, i+1, \dots, n-1$ to become numbered $i+1, i+2, \dots, n$.
- (6) **Delete** the element at position i , $0 \leq i < n$, causing elements numbered $i+1, i+2, \dots, n-1$ to become numbered $i, i+1, \dots, n-2$.

How to **represent an ordered list?**

Sequential mapping (Use **array**)

- Associate the list **element a_i** with the array **index i**
- **Complexity:**
 - ✓ Random access any element --- **$O(1)$** .
 - ✓ Operations (5) and (6) ---Data movement **$O(n)$**

Nonsequential mapping---linked representation

A problem solved using sequential list (array)----Polynomials

- ✓ The problem calls for building an **ADT Polynomial** for the representation and manipulation of polynomials in a single variable (say x).
- ✓ Two polynomials examples:

$$A(x)=3x^2+ 2x+4$$

$$B(x)=x^4+ 10x^3+ 3x^2+1$$

ADT 2.3: ADT Polynomial

```
class Polynomial {
```

```
    //  $p(x)=a_0x^{e_0}+\dots+a_nx^{e_n}$  ; a set of ordered pairs of  $\langle e_i, a_i \rangle$ ,
```

```
    // where  $a_i$  is a nonzero float coefficient and  $e_i$  is a non-negative exponent
```

```
public:
```

```
    Polynomial ( );    // Construct the polynomial  $p(x)=0$ 
```

```
    void AddTerm (Exponent e, Coefficient c);
```

```
    // add the term  $\langle e,c \rangle$  to *this, so that it can be initialized
```

```
    Polynomial Add (Polynomial poly);
```

```
    // return the sum of the polynomials *this and poly
```

```
    Polynomial Mult (Polynomial poly);
```

```
    // return the product of the polynomials *this and poly
```

```
    float Eval ( float f); // evaluate polynomial *this at f and return the result
```

```
}
```

Polynomial Representation-using array

Some assumptions and definitions:

- ✓ Assume the polynomial is in the form as: $a(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$.
- ✓ Note: The coefficients are stored in **decreasing order of exponents**.
- ✓ The **degree of a polynomial** is the largest exponent from among the nonzero terms.

Representation 1

- Defining the **private data members** of Polynomial as follows:

private:

```
int degree; // degree  $\leq$  MaxDegree
```

```
//MaxDegree is a constant that represents the largest-degree  
//polynomial that is to be represented.
```

```
float coef[MaxDegree+1]; //coefficient array
```

- Assume a is a Polynomial class object, then the polynomial $a(x)$ would be represented as: $a.degree = n$; $a.coef[i] = a_{n-i}$, $0 \leq i \leq n$
- When $a.degree \ll \text{MaxDegree}$, this representation is **very wasteful in memory use**.

Representation 2

- To improve, we define **coef** as a data member so that its size is **a.degree+1**. This can be done by declaring the following private data members.

private:

int degree;

float *coef;

- and **adding the following constructor** to Polynomial:

```
Polynomial::Polynomial(int d)
```

```
{
```

```
    int degree=d;
```

```
    coef= new float[degree+1];
```

```
}
```

Representation 3

- Representation 2 is **still not desirable for sparse polynomial**.
- For instance, the polynomial $x^{1000}+1$ **makes 999 entries of the coef be zero**, if representation 2 is used.
- To overcome this problem, we store only **the nonzero terms**.
- Then $a(x)$ is in the form: $a(x) = b_m x^{e_m} + b_{m-1} x^{e_{m-1}} + \dots + b_0 x^{e_0}$, where **$b_i \neq 0$, $e_m > e_{m-1} > \dots, e_0 \geq 0$**
- Now, we should store the **coefficient and exponent** of each nonzero term at the same time.

- So, we define a class **Term** as below:
`class Polynomial; // forward declaration`
`class Term {`
`friend class Polynomial;`
`private:`
`float coef; // coefficient`
`int exp;    // exponent`
`};`
- The private data number of **Polynomial** are defined as follows:

```
class Polynomial {  
public:  
    .....  
private:  
    Term *termArray;  
    int capacity; // size of termArray  
    int terms; // number of nonzero terms  
}
```

Comparing representation 3 with representation 2

For $a(x) = 2x^{1000} + 1$:

In representation 3, for a Polynomial class object “a”,
`a.termArray` looks like:

coef	2	1		
exp	1000	0		

which uses only **6 units of spaces** (one for `A.capacity`, one for `A.terms`, two for coefficients, two for the exponents).

But **1002 units of spaces** (one for degree, and 1001 for coefficients) are required in representation 2.

Many zero terms --- Representation 3 is good

Few zero terms --- Representation 3 is not very good.

Representation 3 may use about twice as much space as does presentation 2 when there is no zero terms.

Polynomial Addition

We use **representation 3** to obtain $c = a + b$.

Basic idea and conventions:

- ✓ Because the exponents are in **descending order**, we can use the **merging idea** to add $a(x)$ and $b(x)$ term by term to produce $c(x)$.
- ✓ Assume that the **default constructor** for Polynomial initializes **capacity and terms** to 1 and 0 respectively.
- ✓ **An new term of c** are inserted into its termArray by calling function **NewTerm**.
- ✓ If the space in termArray of c is not enough, its **capacity is doubled**.

```
1 Polynomial Polynomial::Add (Polynomial b)
2 { // return the sum of the polynomials *this and b.
3   Polynomial c;
4   int aPos=0, bPos=0;
5   while (( aPos < terms) && (bPos < b.terms))
6     if (termArray[aPos].exp==b.termArray[bPos].exp) {
7       float t = termArray[aPos].coef + termArray[bPos].coef
8       if ( t ) c.NewTerm (t, termArray[aPos].exp);
9       aPos++; bPos++;
10    }
12    else if (termArray[aPos].exp < b.termArray[bPos].exp) {
13      c.NewTerm (b.termArray[bPos].coef, b.termArray[bPos].exp);
14      bPos++;
15    }
```

```
15  else {
16      c.NewTerm (termArray[aPos].coef, termArray[aPos].exp);
17      aPos++;
18  }
19  // add in the remaining terms of *this
20  for ( ; aPos < terms; aPos++ )
21      c.NewTerm(termArray[aPos].coef, termArray[aPos].exp );
22  // add in the remaining terms of b
23  for ( ; bPos < b.terms; bPos++ )
24      c.NewTerm(b.termArray[bPos].coef, b.termArray[bPos].exp);
25  return c;
26 }
```

```

void Polynomial::NewTerm(const float theCoeff,
                           const int theExp)
{ // add a new term to the end of termArray.
  if (terms == capacity)
  { // double capacity of termArray
    capacity *= 2;
    term *temp = new term[capacity]; // new array
    copy(termArray, termArray + terms, temp);
    delete [ ] termArray; // deallocate old memory
    termArray = temp;
  }
  termArray[terms].coef = theCoeff;
  termArray[terms++].exp = theExp;
}

```

Analysis of Add:

Let m, n be the number of nonzero terms in a and b respectively.

- line 3 and 4--- $O(1)$
- in each iteration of the **while loop**, $aPos$ or $bPos$ or both increase by 1, while loop terminates when either $aPos$ equals $a.terms$ or $bPos$ equals $b.terms$, **the number of iterations of this loop $\leq m+n-1$**
- if ignore the time for doubling the capacity, each iteration takes $O(1)$
- line 20--- $O(m)$, line 23--- $O(n)$
- So the asymptotic computing time for Add:
 $O(m+n+\text{time spent in array doubling})$

Analysis of array doubling:

- We assume that it takes $O(1)$ time to allocate memory and that each invocation of the constructor of Term and each element copy takes $O(1)$ time. Hence the time for **doubling an array is linear in the size of new array**
- initially, c.capacity is 1
- suppose when Add terminates, c.capacity is 2^k
- the total time spent over all array doubling is
$$O\left(\sum_{i=1}^k 2^i\right) = O(2^{k+1}) = O(2^k)$$
- since **c.terms** $> 2^{k-1}$ and **m + n** $\geq$ **c.terms**, then **$2^k \leq 2(m + n)$** , thus the total time for array doubling is
$$O(\text{c.terms}) = O(m + n)$$

- so, even with the time spent in array doubling, the total run time of Add is $O(m + n)$.
- experiments indicate that array doubling is responsible for a very small fraction of the total run time of add.

Another problem solved using Array ---- Sparse Matrices

Introduction

A general **matrix** consists of **m** rows and **n** columns (**m** × **n**) of numbers, as shown:

	0	1	2
0	-27	3	4
1	6	82	-2
2	109	-64	11
3	12	8	9
4	48	27	47

Fig.2.2(a) 5×3

	0	1	2	3	4	5
0	15	0	0	22	0	-15
1	0	11	3	0	0	0
2	0	0	0	-6	0	0
3	0	0	0	0	0	0
4	91	0	0	0	0	0
5	0	0	28	0	0	0

Fig. 2.2(b) 6×6

A matrix of $m \times m$ is called a **square**.

A matrix with **many zero entries** is called **sparse**.

Representations for matrix

➤ A natural way ----

- a two-dimensional array: $a[m][n]$
- access to an element $a[i][j]$ easily.
- **But, for sparse matrix, wasteful of both memory and time.** Because every element including zero element is stored and should be managed and dealt with.

➤ Alternative way ----

- **store nonzero elements** explicitly.

Abstract data type SparseMatrix

class SparseMatrix

```
{ // a set of <row, column, value>, where row, column are  
  // non-negative integers and form a unique combination;  
  // value is also an integer.
```

public:

```
  SparseMatrix ( int r, int c, int t);
```

```
  // creates a  $r \times c$  SparseMatrix with a capacity of t nonzero  
  // terms
```

```
  SparseMatrix Transpose ( );
```

```
  // return the SparseMatrix obtained by transposing *this
```

```
  SparseMatrix Add ( SparseMatrix b);
```

```
  SparseMatrix Multiply ( SparseMatrix b);
```

```
};
```

Sparse Matrix Representation

- ✓ We represent an element by using **triple** $\langle \text{row}, \text{col}, \text{value} \rangle$, so we define the class **MatrixTerm** as below

```
class SparseMatrix;
class MatrixTerm {
friend class SparseMatrix;
Private:
    int row, col, value;
};
```
- ✓ **An array of triples** to represent a sparse matrix.
- ✓ **Required:** these triples are sorted in **ascending order** by $\langle \text{row}, \text{col} \rangle$.

✓ We need also

the number of **rows**

the number of **columns**

the number of **nonzero elements**

✓ Thus the **number variables** in SparseMatrix is:
private:

```
int rows, cols, terms, capacity;
```

```
//terms is the total number of nonzero entries
```

```
//capacity is the size of smArray
```

```
MatrixTerm *smArray;
```

Now we can store the matrix of Fig.2.2 (b) as Fig.2.3 (a).

						smArray	row	col	value
						[0]	0	0	15
						[1]	0	3	22
0	1	2	3	4	5	[2]	0	5	-15
15	0	0	22	0	-15	[3]	1	1	11
0	11	3	0	0	0	[4]	1	2	3
0	0	0	-6	0	0	[5]	2	3	-6
0	0	0	0	0	0	[6]	4	0	91
91	0	0	0	0	0	[7]	5	2	28
0	0	28	0	0	0				

Fig.2.3 (b)

Fig.2.3 (a)

Transposing a Matrix

Transpose:

If an element is at position $[i][j]$ in the original matrix, then it is at position $[j][i]$ in the transposed matrix.

How to transpose a matrix when it is represented by an array of triples?

First idea:

For (each row i)

store element (i, j, value)
of the original matrix as (j, i, value) of the transpose;

Difficulty:

NOT knowing where to put (j, i, value) until all other elements that precede it have been processed.

For instance, the first four triples in the original matrix, we encounter:

$(0,0,15)$ which becomes $(0,0,15)$

$(0,3,22)$ which becomes $(3,0,22)$

$(0,5,-15)$ which becomes $(5,0,-15)$

$(1,1,11)$ which becomes $(1,1,11)$

Where to put these elements???

Improvement:

For (**each column j**)

store **(i, j, value)** of the original matrix as **(j, i, value)** of the transpose;

That is, “find all elements in **column 0** and store them in **row 0**, find all elements in column 1 and store them row 1, etc.”

Since the **rows are originally in order**, we will locate elements in the **correct column order**.

For instance, the first four triples **in the column order**:

(0,0,15) becomes (0,0,15), put in the **1st position**.

(4,0,91) becomes (0,4,91), put in the **2nd position**.

(1,1,11) becomes (1,1,11), put in the **3rd position**.

(1,2,3) becomes (2,1,3), put in the **4th position**.

It's OK!

smArray	row	col	value		smArray	row	col	value
[0]	0	0	15		[0]	0	0	15
[1]	0	3	22		[1]	0	4	91
[2]	0	5	-15		[2]	1	1	11
[3]	1	1	11		[3]	2	1	3
[4]	1	2	3		[4]	2	5	28
[5]	2	3	-6		[5]	3	0	22
[6]	4	0	91		[6]	3	2	-6
[7]	5	2	28		[7]	5	0	-15

Program 2.10: Transposing a matrix

```
1 SparseMatrix SparseMatrix::Transpose ( )
2 { // return the transpose of *this
3     SparseMatrix b(cols, rows, terms); //capacity of
                                         // b.smArray is terms
4     if (terms > 0)
5     { //nonzero matrix
6         int currentB = 0;
7
8         /*The variable currentB always gives us the position where the
9         next term in the transpose is to be inserted.*/
```

```

7   for ( int c=0; c<cols; c++ ) // transpose by columns
8       for ( int i=0; i<terms; i++ )      /*find all elements in column c
9       // find and move terms in column c and store them in the transpose.*/
10          if ( smArray[i].col == c )
11          {
12              b.smArray[CurrentB].row = c;
13              b.smArray[CurrentB].col = smArray[i].row;
14              b.smArray[CurrentB++].value= smArray[i].value;
15          }
16      } // end of if (terms > 0)
17  return b;
18 }

```

Time complexity of Transpose:

- Lines 3-6 take a constant amount time
- For each iteration of the loop of lines 7 through 15, the condition of line 10 is tested **terms times**.
- The number of iterations of this loop is **cols**. The total time for line 10 is **terms*cols**.
- The assignments of lines 12-14 take place exactly **terms times**.
- Total time for Transpose: **$O(\text{cols} * \text{terms})$**
- The space needed: **$O(1)$**

Think more:

$O(\text{cols} * \text{terms})$ is not always good. If $\text{terms} = O(\text{cols} * \text{rows})$ then it becomes $O(\text{cols}^2 * \text{rows})$ ---too bad!

- Because we scan the array again and again for different column in *this
- But if the matrices is represented by 2-dimensional array, we can get an algorithm with $O(\text{cols} * \text{rows})$ time as:

```
for (int j=0; j < columns; j++)  
    for (int i=0; i < rows; i++) B[j][i] = A[i][j];
```

Further improvement:

- If we can **avoid rescanning** the array again and again, we can do much better: doing it in **$O(\text{cols} + \text{terms})$** .
- Firstly, we get **the number of elements in each column** of ***this** (= the number of elements in each row of b);
- Hence, the **starting point of each row in b** is easily obtained;
- Thus, we can **move the elements of *this** one by one into their **right position in b**.

Now we can get the algorithm **FastTranspose**.

Program 2.11: Transposing a matrix faster

```
1 SparseMatrix SparseMatrix::FastTranspose ( )
2 { // return the transpose of *this in  $O(\text{terms} + \text{cols})$  time.
3   SparseMatrix b(cols, rows, terms);
4   if (terms > 0)
5   { // nonzero matrix
6     int *rowSize = new int[cols];
7     int *rowStart = new int[cols];
8     /* lines 9 and 10 compute rowSize[i] which stores the number
       of terms in row i of b (col i of *this) */
9     fill(rowSize, rowSize + cols, 0); // initialize
10    for (i=0; i<terms; i++ ) rowSize[smArray[i].col]++;
```

```
11  /* lines 12 and 13 compute rowStart[i] which is the starting
    position of row i in b*/
12  rowStart[0] = 0;
13  for (i=1;i<cols;i++) rowStart[i]=rowStart[i-1]+rowSize[i-1];
14  for (i=0; i<terms; i++)
15  {    // copy from *this to b
16      int j = rowStart[smArray[i].col]; // the inserting position
17      b.smArray[j].row = smArray[i].col;
18      b.smArray[j].col = smArray[i].row;
19      b.smArray[j].value = smArray[i].value;
20      rowStart[smArray[i].col]++;
21  } // end of for
```

```
22  delete [ ] rowSize;  
23  delete [ ] rowStart;  
24  } // end of if  
25  return b;  
26  }
```

smArray	row	col	value		smArray	row	col	value
[0]	0	0	15		[0]	0	0	15
[1]	0	3	22		[1]	0	4	91
[2]	0	5	-15		[2]	1	1	11
[3]	1	1	11	→	[3]	2	1	3
[4]	1	2	3		[4]	2	5	28
[5]	2	3	-6		[5]	3	0	22
[6]	4	0	91		[6]	3	2	-6
[7]	5	2	28		[7]	5	0	-15

After line 13, we get :

	[0]	[1]	[2]	[3]	[4]	[5]
RowSize=	2	1	2	2	0	1
RowStart=	0	2	3	5	7	7

Note the error in P101 of the text book!

Analysis of FastTranspose:

There are 3 loops:

- line 10--- $O(\text{terms})$
- line 13--- $O(\text{cols})$
- line 14 – 21--- $O(\text{terms})$
- and line 9--- $O(\text{cols})$, other lines--- $O(1)$

Total time: $O(\text{cols} + \text{terms})$

The computing time of $O(\text{cols} + \text{terms})$ becomes $O(\text{cols} * \text{rows})$ when terms is $O(\text{cols} * \text{rows})$.

The String Abstract Data Type

A string has the form: $S = s_0, s_1, \dots, s_{n-1}$, where $s_i \in \text{char}$, $0 \leq i < n$, n is the length. If $n=0$, then S is an **empty or null** string.

ADT 2.5: Abstract data type **String**

class **String**

{ // a finite set of zero or more characters;

public:

String (**char** *init, **int** m);

// Constructor that initializes ***this** to string init of length m

```
bool operator == (String t );
```

```
// if *this equals t, return true else false.
```

```
bool operator ! ( );
```

```
// if *this is empty return true else false.
```

```
int Length ( );
```

```
// return the number of chars in *this
```

```
String Concat (String t);
```

```
String Substr (int i, int j);
```

```
// return a string containing the j characters of *this at position i
```

```
int Find (String pat);
```

```
// return an index i such that pat matches the substring of *this
```

```
// that begins at position i. Return -1 if pat is either empty or not
```

```
// a substring of *this.
```

```
};
```

Representation of String

We can assume in the **String class**, strings are represented by the **private data member str** of **type char ***:

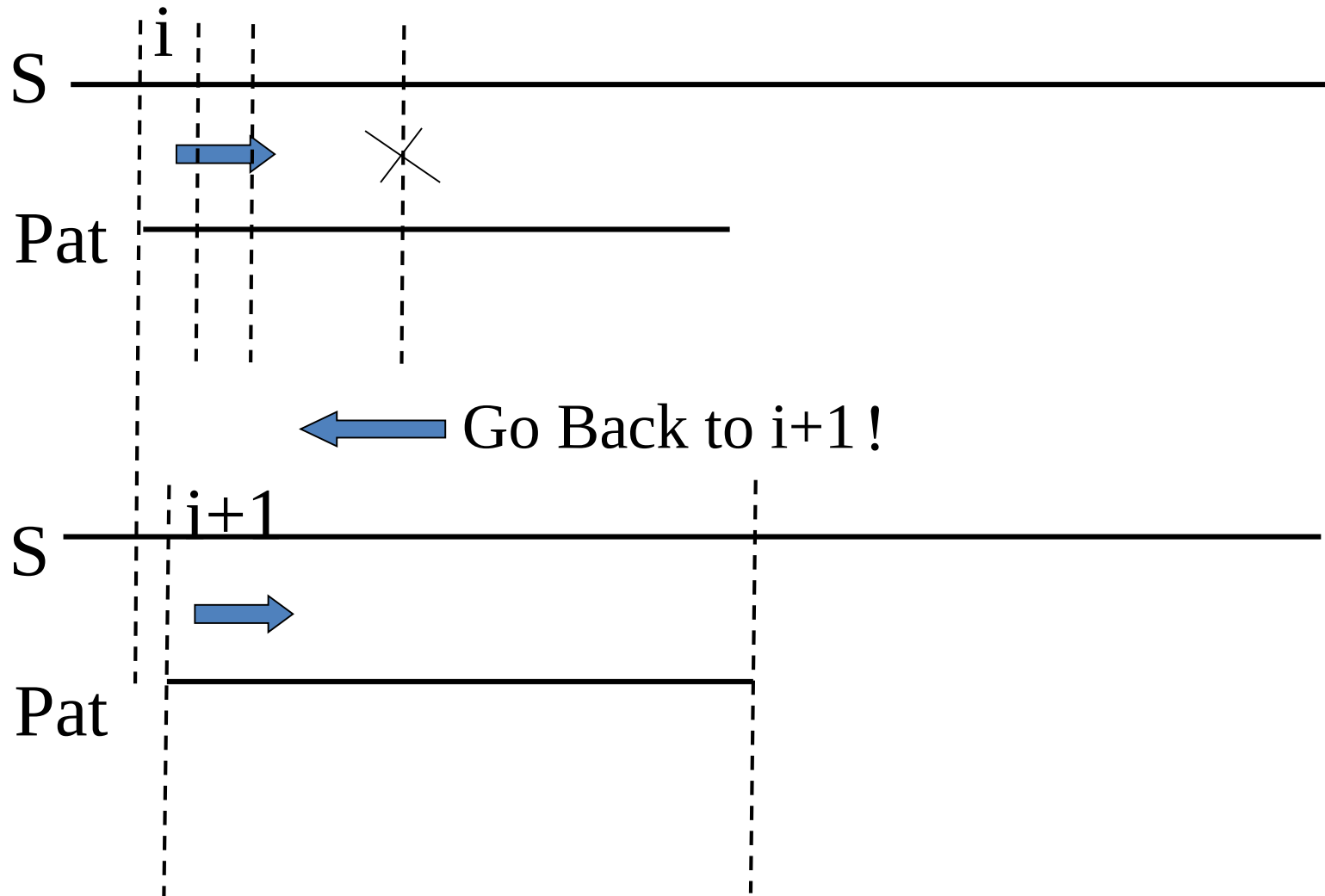
private:

char* str;

2.6.1 String Pattern Matching: A Simple Algorithm

String pattern matching problem: Assume that we have two strings **s** and **pat**, where **pat** is a **pattern** to be searched for in **s**. We will **determine if pat is in s** by using the function **Find**.

String Pattern Matching: A Simple Algorithm



```

int String::Find ( String pat )
{ // Return -1 if pat does not occur in *this; otherwise
  // return the first position in *this, where pat begins.
  if (pat.Length( ) == 0) return -1; // pat is empty
  for (int start=0; start<=Length( ) - pat.Length( ); start++)
  { // check for match beginning at str[start]
    for (int j=0; j<pat.Length( )&&str[start+j]==pat.str[j];j++);
    if (j== pat.Length( )) return start; // match found
    // no match at position start
  }
  return -1;    // pat does not occur in s
}

```

Program 2.15: Exhaustive pattern matching.

The complexity of it is $O(\text{LengthP} * \text{LengthS})$.

Problem: Search will move backwards (rescan) in the string s if a mismatch occurs.

2.6.2 String Pattern Matching: The Knuth-Morris-Pratt Algorithm

- Can we get an algorithm which **avoid rescanning in the string s** and thus improve the performance?
- Actually, if we can go **without moving backwards in the string s** , it is likely to do the job in **$O(\text{LengthP} + \text{LengthS})$** .
- **Knuth, Morris, and Pratt** have developed a pattern-matching algorithm that works in this way and has **linear complexity**. This is **optimal** for this problem, as in the worst it is necessary to look at characters in the pattern and string at least once.

The basic idea of KMP Algorithm

- If a mismatch occurs, we can use **our knowledge of the characters in the pattern and the position in the pattern where the mismatch occurred** to determine where we should continue the search.

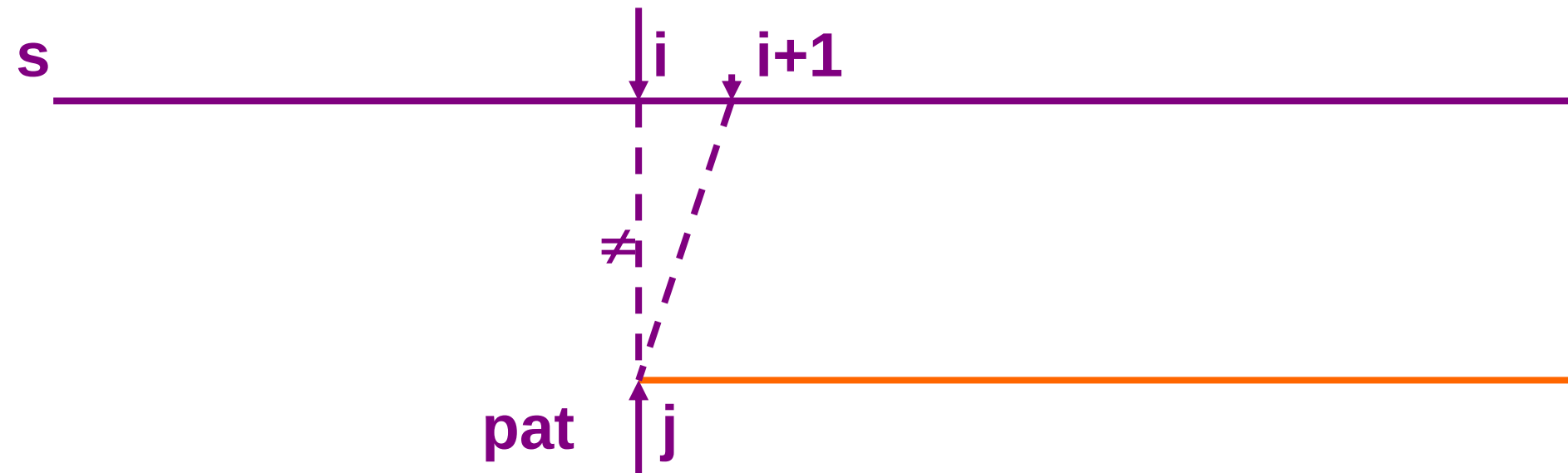
- Now we show the idea with an example.

Let $s = s_0s_1 \dots s_{m-1}$, $pat = abcabbacab$.

Assume that we are **currently** determining whether there is a match **at** s_i .

$s = s_0 s_1 \dots s_{m-1}$, $pat = \text{abcabbacab}$

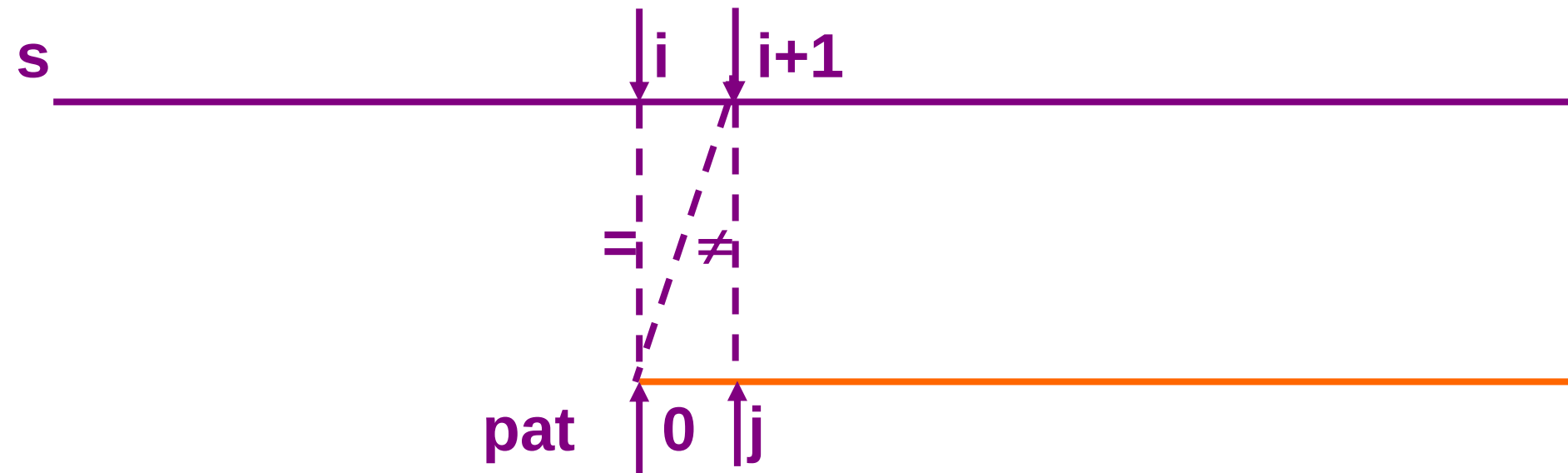
(1) If $s_i \neq p_0(a)$, then we proceed by comparing s_{i+1} and $p_0(a)$;



case: $j = 0$

$s = s_0 s_1 \dots s_{m-1}$, $pat = \text{ab}cabbacab$

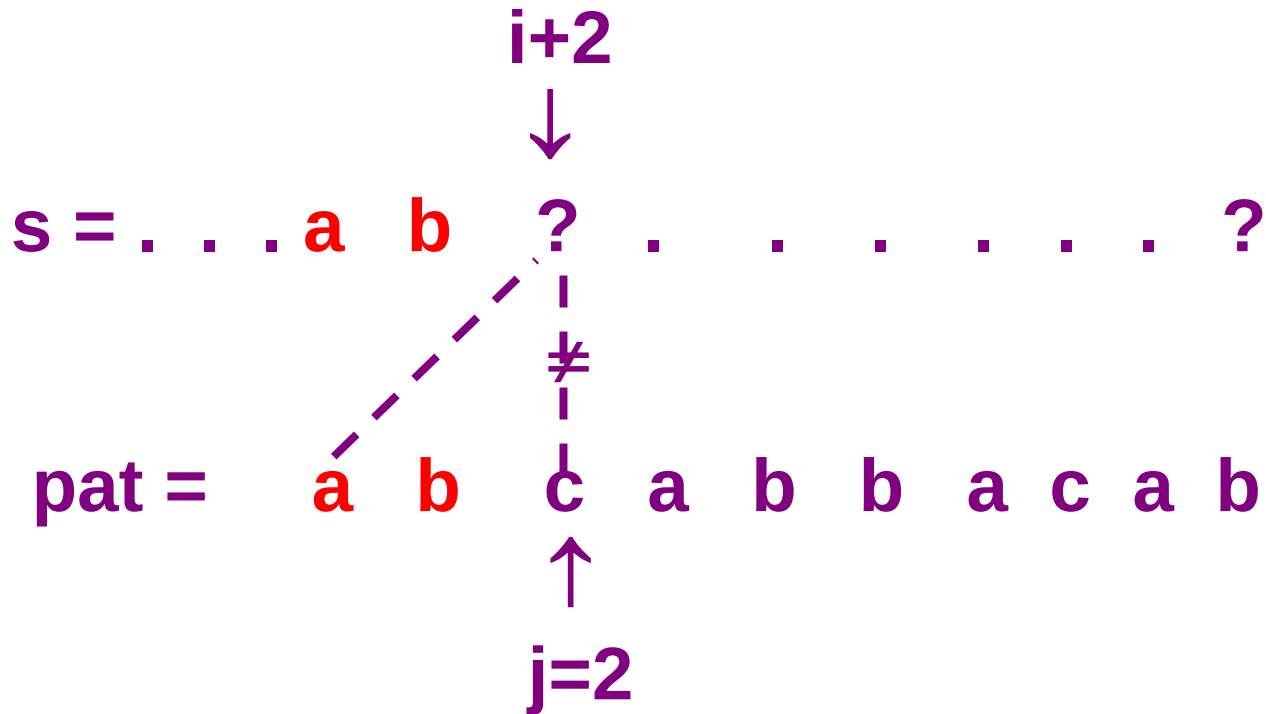
(2) If $s_i = p_0$ (a) and $s_{i+1} \neq p_1$ (b), then we proceed by comparing s_{i+1} and p_0 (a).



case: $j = 1$

$s = s_0 s_1 \dots s_{m-1}$, $pat = abcabbacab$

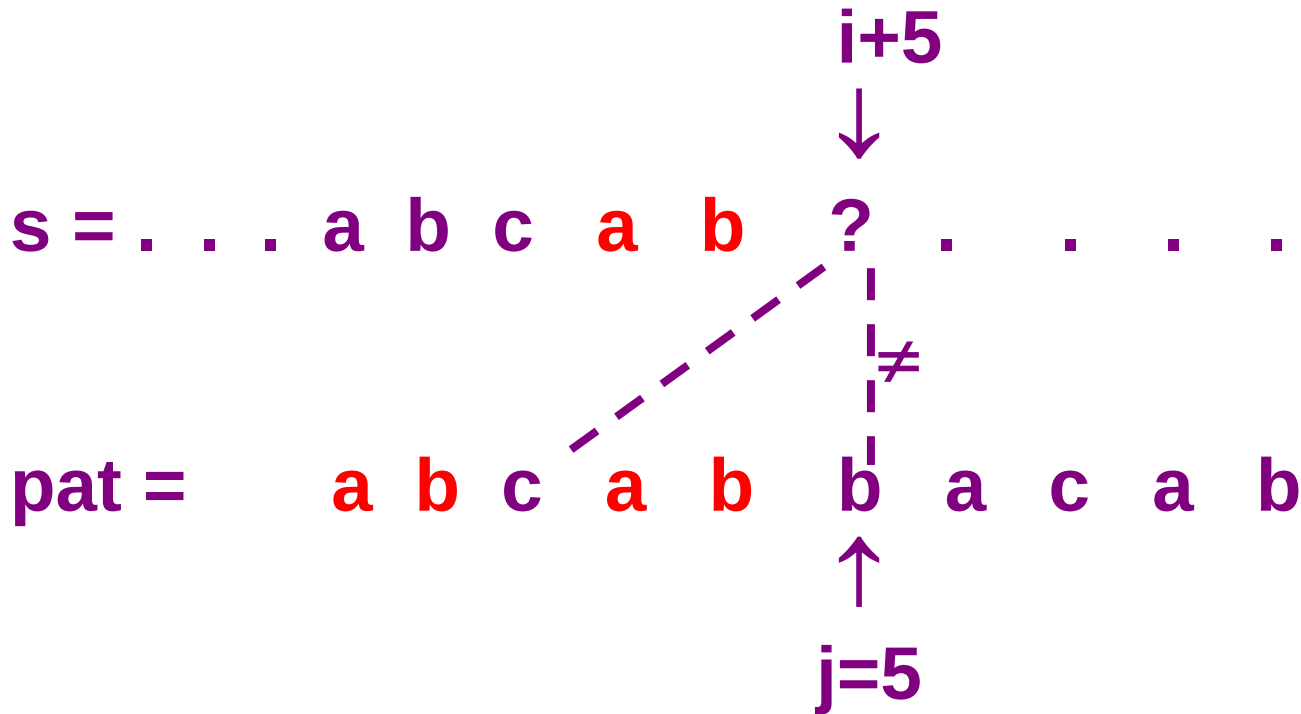
(3) other situations (case 3: $j > 1$):



Here $s_{i+2} \neq p_2$ (c), we can proceed by comparing the **first character p_0 (a) in pat with s_{i+2}** . There is no need to compare p_0 (a) in pat with s_{i+1} , since we already know that s_{i+1} is the same as the second character of pat , p_1 (b), but $p_1 \neq p_0$.

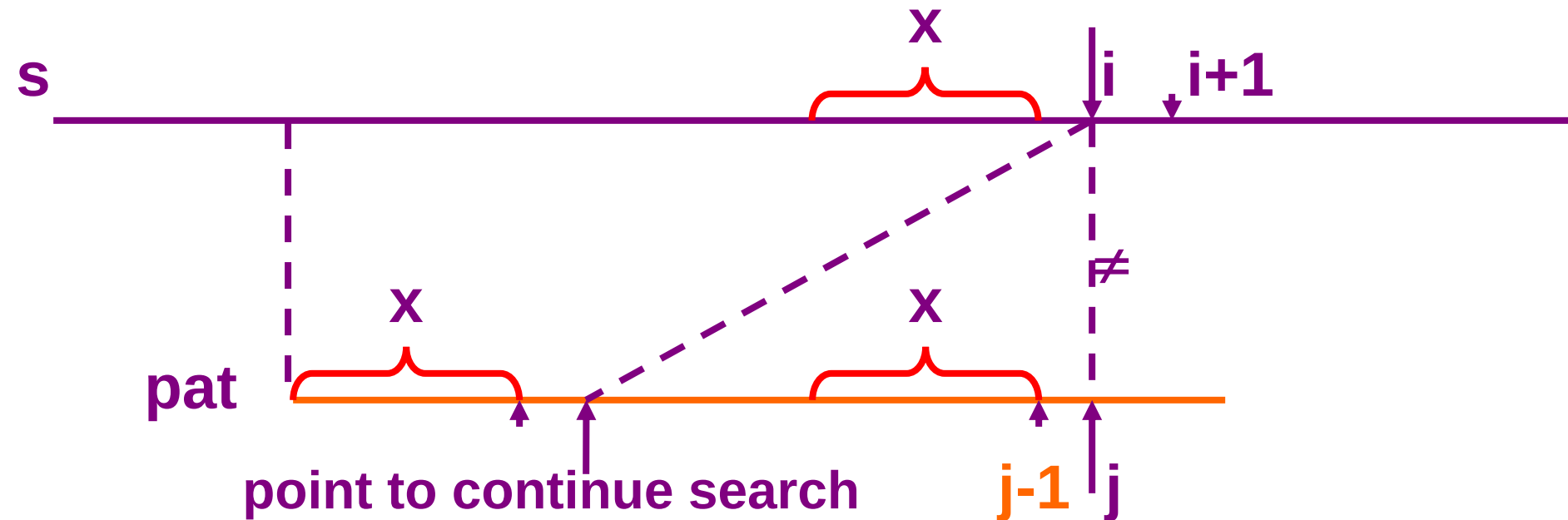
$s = s_0 s_1 \dots s_{m-1}$, $pat = abcabcacab$

(3) other situations (case 3: $j > 1$):



Here, $s_{i+5} \neq p_5$ (b), we proceed by comparing s_{i+5} and p_2 (c), since we know $p_0 p_1 = p_3 p_4 = s_{i+3} s_{i+4}$.

In fact, according to the position in the pattern where a mismatch occurs with a character in s , we can determine where in the pattern to continue the search for a match without moving backwards in s .



case: $j \neq 0$

Failure Function

To formalize the above idea, we define a **failure function** for a pattern.

Definition: if $p = p_0 p_1 \dots p_{n-1}$ is a pattern, then its **failure function** f , is defined as:

$$f(j) = \begin{cases} \text{largest } k < j, \text{ such that } \overset{\text{prefix}}{p_0 p_1 \dots p_k} = \overset{\text{postfix}}{p_{j-k} p_{j-k+1} \dots p_j} \\ \text{if such a } k \geq 0 \text{ exists} \\ -1 \text{ otherwise (without the same prefix and postfix)} \end{cases}$$

Understanding Failure Function

Note:

- $f(j)$ is the **longest length** of the same prefix and suffix at position j in the pattern **minus one**.
- largest k : jump farthest but no match be missed
- $k < j$: avoid dead loop

Failure Function Example

For example, pat = **a b c a b c a c a b**, we have

j	0	1	2	3	4	5	6	7	8	9
pat	a	b	c	a	b	c	a	c	a	b
f	-1	-1	-1	0	1	2	3	-1	0	1

Class exercise (P119 7(c)):

- **Compute the failure function for the following pattern: abaabaabb**

Rule for pattern matching

If we have the failure function of the pattern, we can get the following rule for pattern matching (**j and i are the positions in the pattern and in the main string respectively when a mismatch occurs**):

- ✓ If **j=0**, then we continue by comparing **s_{i+1} and p_0** .
- ✓ If **j≠0**, and a partial match is found such that **$s_{i-j} \dots s_{i-1} = p_0 p_1 \dots p_{j-1}$ but $s_i \neq p_j$ ($j > 0$)** then matching continues by comparing **s_i and $p_{f(j-1)+1}$** .

This rule can easily translate to the function **FastFind** (Program 2.16).

Program 2.16: Pattern-matching with a failure function

```
1 int String::FastFind (String pat)
2 { // Determine if pat is a substring of s
3   int PosP = 0, PosS = 0;
4   int LengthP= pat.Length( ), LengthS= Length( );
5   while ((PosP < LengthP) && (PosS < LengthS))
6     if ( pat.str[PosP] == str[PosS] ) { // characters match
7       PosP ++; PosS ++;
8     }
9     else
10      if ( PosP==0) // characters mismatch at position 0 in the pattern
11        PosS++;
12      else PosP= pat.f [PosP-1] + 1; //end of while
13  if ((PosP < LengthP) || LengthP==0)) return -1;
14  else return PosS - LengthP ;
15 }
```

The **failure function** is represented by **an array of integers f**, which is a private data member of String.

Analysis of FastFind:

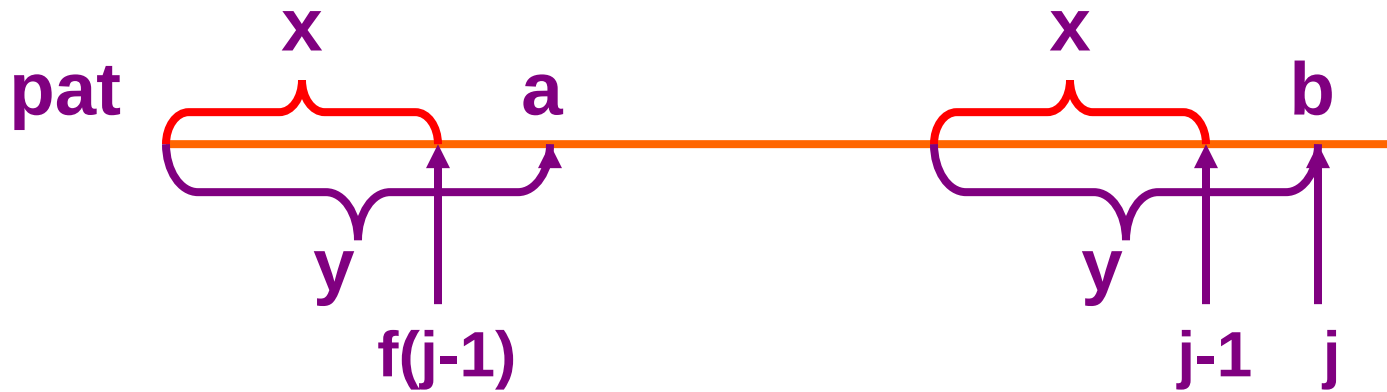
- Lines 7 and 11 --- executed at most **LengthS times**, since in each iteration PosS is increased by 1 but **never decreased**. So PosP can **move right** on pat at most **LengthS times** (line 7).
- Line 12--- PosP **moves left** on pat, it can be executed at most **LengthS times**, otherwise PosP becomes less than 0. Note that $0 \leq f(\text{posP}-1)+1 < \text{posP}$.
- Consequently, the maximum number of iterations of the while loop is **LengthS**, and the computing time of FastFind is **$O(\text{LengthS})$** if we have the f array.

Problems:

- How to compute the failure function?
- What is the computing time of the failure function?

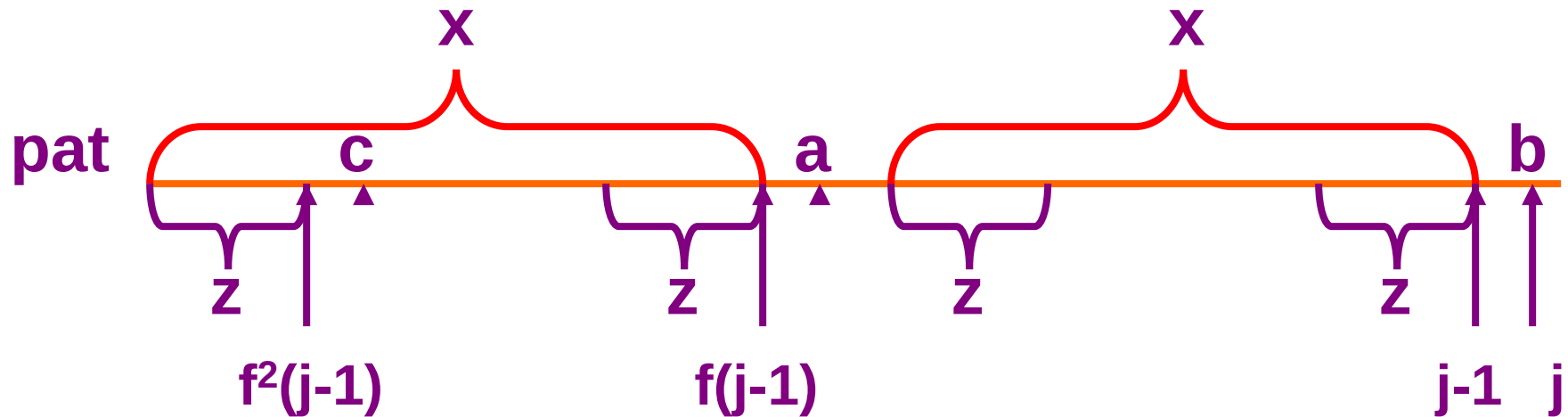
How to compute the failure function (f array)?

- $f(0) = -1$
- For $j > 0$, if we have $f(j-1)$, we can compute $f(j)$ from it by the following observation:



If **a=b** ($p_{f(j-1)+1} = p_j$), then $f(j) = f(j-1) + 1$

Else:



If $c=b$ ($p_{f^2(j-1)+1} = p_j$), Then $f(j)=f(f(j-1))+1=f^2(j-1)+1$

Else

In general, we have the following **restatement of the failure function**:

$$f(j) = \begin{cases} -1 & \text{if } j=0 \\ f^m(j-1)+1 & \text{where } m \text{ is the least } k \text{ for which} \\ & \mathbf{P_{f^k(j-1)+1} = P_j} \\ -1 & \text{if there is no } k \text{ satisfying the above} \end{cases}$$

Now we get the algorithm to compute f .

Program 2.17: Computing the failure function

```
1 void String::Failurefunction( )
2 { // compute the failure function of the pattern *this.
3   int LengthP= Length( );
4   f [0]= -1;
5   for (int j=1; j< LengthP; j++) // compute f[1], f[2]...in order
6   {
7     int i=f [j-1];
8     while ((* (str+j)!=*(str+i+1)) && (i>=0)) i=f[i];
9     //try for f2(j-1), f3(j-1)...
10    if ( *(str+j)==*(str+i+1))
11      f[j]=i+1;
12    else f[j]= -1; //i<0
13  } //end of for
14 }
```

Analysis of FailureFunction:

- ✓ i is set to -1 when the for loop is executed at the first time (we call it the initial value of i).
- ✓ At the beginning of each for loop:
 - i is either reset to -1 (when $j=1$ or the previous iteration went through line 11),
 - or it is reset to a value 1 greater than its terminal value on the previous iteration (when the previous iteration went through line 10).
- ✓ Only $\text{LengthP} - 1$ executions of line 7 are made, so the value of i has a total increment of at most $\text{LengthP} - 1$ (through line 11) .

- ✓ **i decreases** (line 8) in each iteration of the while, or i is either reset to **-1**, but **i is always not less than the initial value**, so i **cannot be decremented more than LengthP - 1** times.
- ✓ Consequently, the while loop is iterated **at most LengthP - 1 times** over the whole algorithm.
- ✓ So, the computing time of FailureFunction is **$O(\text{LengthP})$** .

Therefore:

When the failure function is **not known** in advance, pattern matching can be carried out in time **$O(\text{LengthP} + \text{LengthS})$** by first computing the failure function and then performing a pattern match using the FastFind.

Exercise(s) and Experiment(s)

Exercise(s):

- **P93:6; P107:1; P119:7,9**

Experiment(s):

- **P93:2; P107:4; P118:1; P120:1**